



SIMATS
ENGINEERING



SIMATS
Saraswathi Institute of Medical and Technical Sciences
Affiliated to - Coimbatore University under Section 3 of UAC Act 1956

B.E / B.Tech

WEB TECHNOLOGY

Course Code: ITA02

ASSIGNMENT 1 — SCENARIO-BASED ASSIGNMENT

**Design and Development of an
Interactive University Course Registration Portal**

Submitted by

Name: __GAYATHRI _K

Register No.: __192421319__

Department / Section: __B.Tech IT__

Date of Submission: __01/09/2026__

1. Introduction

This report documents the design and development of a web-based Course Registration Portal for a university, built using core Unit I (HTML/XHTML) and Unit II (CSS and JavaScript) concepts. The portal presents course information to students, collects registration details through a form, validates that data entirely in the browser, and generates a live registration summary — including the total number of selected courses and total credits — without reloading the page.

The application is delivered as a single self-contained HTML file (course-registration-portal.html) containing the markup, styling, and client-side script, so it can be opened directly in any browser without a server.

1.1 Scenario Recap

A university wants a simple portal where students can view available courses and submit basic registration details. The portal must follow proper HTML/XHTML structure, provide clear navigation and organized content, offer a professional responsive interface, validate registration data on the client side, and calculate/display a registration summary dynamically.

1.2 Course Outcomes Addressed

- CO I: Create web pages using HTML/XHTML, apply proper client-side structure — addressed through semantic tags, lists, tables, hyperlinks, and the registration form.
- CO II: Apply CSS layout techniques (Flexbox/Grid), responsive design, and JavaScript for validation, functions, arrays/objects, and debugging — addressed through the stylesheet and the registration script.

2. Objectives

1. Build a semantically structured webpage with a university title, navigation, course information, and contact details.
2. Display at least five available courses in an HTML table with Course Code, Course Name, Credits, and Course Type.
3. Use lists for registration instructions/eligibility and provide internal and external hyperlinks.

4. Create a student registration form capturing Register Number, Student Name, Email, Department, Semester, and Course Selection.
5. Apply CSS selectors, the box model, Flexbox/Grid, responsive design, hover effects, and spacing for a professional interface.
6. Validate mandatory fields, email format, and semester range using JavaScript.
7. Store selected courses in a JavaScript array of objects and display them dynamically.
8. Write a reusable function to calculate total selected courses and total credits.
9. Render a registration summary (name, register number, department, courses, credits) without a page reload.
10. Use the browser console to find and fix at least one issue during development.

3. Tools and Technologies Used

- HTML5 — page structure, semantic sectioning, table, lists, form controls
- CSS3 — selectors, box model, Flexbox, Grid, media queries, hover/focus states
- JavaScript (ES5/ES6, vanilla) — form validation, DOM manipulation, arrays/objects, functions
- Browser Developer Tools — console logging and debugging during development
- Google Fonts (Lora for headings, Inter for body text) for typography

4. Design Approach

The portal represents a fictional institution, Riverbend University, so that every design decision — palette, type, and copy — has real subject matter behind it rather than generic placeholder styling.

4.1 Colour and Type

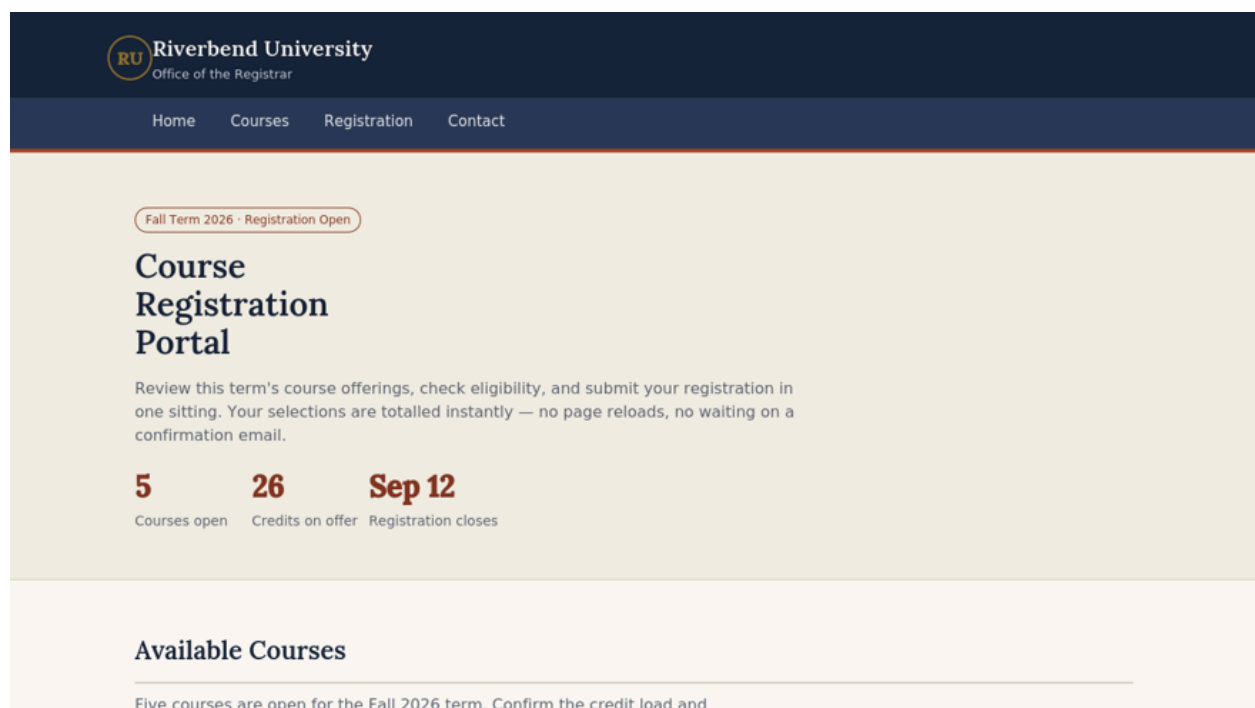
- Ink navy (#182338) and a muted brick-red accent (#A4442E) evoke an academic crest/registrar's-office feel, on a warm paper background (#FAF8F3).

- Lora (serif) is used for headings to suggest an institutional, printed-prospectus tone; Inter (sans-serif) is used for body text and form labels for legibility.

4.2 Layout

- A fixed header holds the university crest/title, with a secondary navigation bar (Home | Courses | Registration | Contact) directly beneath it.
- Flexbox is used for the header and hero content; CSS Grid is used for the two-column instructions/links section, the registration form fields, and the summary panel's key-value grid.
- A single media query (max-width: 780px) collapses all grid layouts to one column and stacks form action buttons for mobile use.

5. Implementation



5.1 Page Structure and Navigation

The page uses semantic HTML5 elements — header, nav, main, section, footer, and address — instead of generic divs, so the document outline is meaningful on its own. The navigation bar links to in-page sections using fragment identifiers (#home, #courses, #registration, #contact),

and the hover state on each link is a bottom border transition rather than a colour swap alone, keeping it visible for keyboard focus too.

Fig. 1 — Header, navigation, and hero section of the portal

5.2 Course Information Table

Five courses are listed in a table with a caption, scoped column headers, and a badge-styled Course Type column (Core / Elective / Lab) implemented with small inline spans and CSS colour coding. Zebra-stripping and a hover highlight are applied via CSS selectors (:nth-child(even), :hover).

Course Code	Course Name	Credits	Course Type
ITA02	Web Technology	4	Core
CSA05	Data Structures & Algorithms	4	Core
CSE21	Database Management Systems	4	Core
ITE14	Cloud Computing Fundamentals	3	Elective
CSL08	Web Technology Laboratory	1	Lab

Below the table, an ordered list gives the five registration steps, and a definition-style list shows eligibility figures (minimum attendance, maximum credits, backlogs allowed). A linked list provides one internal hyperlink (back to the registration form), one external hyperlink (UGC guidelines, opened in a new tab), and a mailto: link to the registrar.

Fig. 2 — Course table, registration steps, and eligibility/links panel

Available Courses

Five courses are open for the Fall 2026 term. Confirm the credit load and course type before adding a course to your registration below.

Fall 2026 · Department of Computer Science & Engineering			
Course Code	Course Name	Credits	Course Type
ITA02	Web Technology	4	Core
CSA05	Data Structures & Algorithms	4	Core
CSE21	Database Management Systems	4	Core
ITE14	Cloud Computing Fundamentals	3	Elective
CSL08	Web Technology Laboratory	1	Lab

How to register

1. Confirm your department and current semester with your advisor.
2. Check that prerequisite courses from the previous semester are cleared.
3. Select one or more courses below, keeping total credits within the 22-credit cap.
4. Fill in the registration form with your official register number and university email.
5. Review the generated summary carefully before your advisor countersigns it.

Eligibility & resources

Minimum attendance	75%
Maximum credits / term	22
Backlog courses allowed	2

Useful links

[Jump to the registration form](#)

5.3 Registration Form

The form captures Register Number, Student Name, Email, Department (select), Semester (select), and Course Selection (checkboxes grouped in a fieldset/legend). Each field has an associated `<label>` for accessibility, and inline `<small>` elements reserve space for validation messages so the layout does not jump when an error appears.

Fig. 3 — Student registration form with course-selection checkboxes

[Jump to the registration form](#)
[UGC academic guidelines \(external\)](#)
[Email the Registrar's office](#)

Student Registration

Fields marked * are required. The form is checked in your browser before it is processed, and your summary appears below without reloading the page.

Register Number *
e.g. 21ITR045

Student Name *
e.g. Ananya Rao

University Email *
name@riverbend.edu

Department *
Select department

Semester *
Select semester

Course Selection * (choose at least one)
☐ITA02 — Web Technology (4 cr) ☐CSA05 — Data Structures & Algorithms (4 cr)
☐CSE21 — Database Management Systems (4 cr) ☐ITE14 — Cloud Computing Fundamentals (3 cr)
☐CSL08 — Web Technology Laboratory (1 cr)

Submit registration **Clear form**

5.4 Client-Side Validation (JavaScript)

Validation runs entirely in the browser on form submit, before any summary is generated. The following checks are performed:

- Register Number and Student Name must not be empty.
- Email must match a simple local@domain.tld pattern (isValidEmail).
- Semester must be an integer between 1 and 8 (isValidSemester).
- Department must be selected.
- At least one course checkbox must be checked.

If any check fails, the corresponding field's error message is set, the form submission is stopped with `event.preventDefault()`, and the current input snapshot is written to the console via `console.log()` for debugging.

```
function isValidEmail(value) {  
  
    var pattern = /^[^\s@]+@[^\s@]+\.[a-zA-Z]{2,}$/;  
  
    return pattern.test(value);  
  
}
```

```
function isValidSemester(value) {  
  
    var n = Number(value);  
  
    return value !== "" && Number.isInteger(n) && n >= 1 && n <= 8;  
  
}
```

5.5 Storing Selections and Calculating Totals

Selected courses are collected into an array of plain JavaScript objects, each holding a code, name, and credits value, read from the checkbox's value attribute (encoded as code|name|credits). A separate, reusable function then reduces that array into the totals the summary needs:

```
function collectSelectedCourses() {  
  
    var checks = document.querySelectorAll('input[name="course"]:checked');  
  
    var list = [];  
  
    checks.forEach(function (chk) {  
  
        var parts = chk.value.split("|");  
  
        list.push({ code: parts[0], name: parts[1], credits: Number(parts[2]) });  
  
    });  
  
    return list;  
  
}
```

```
}
```

```
function calculateTotals(courses) {  
  var totalCourses = courses.length;  
  var totalCredits = courses.reduce(function (sum, c) {  
    return sum + c.credits;  
  }, 0);  
  return { totalCourses: totalCourses, totalCredits: totalCredits };  
}
```

calculateTotals() is written independently of the DOM so it is reusable — it is called once on every successful submission, but could equally be called whenever a checkbox changes to show a live running total.

5.6 Dynamic Registration Summary

On successful validation, renderSummary() writes the student's name, register number, department, semester, selected course list, and totals into a summary panel that starts hidden (display: none) and is revealed by toggling a CSS class. Because this happens through direct DOM updates instead of a form action, the page never reloads and the rest of the form state is preserved.

Fig. 4 — Completed form and the resulting dynamic registration summary

Register Number *
23ITR112

Student Name *
Ananya Rao

University Email *
ananya.rao@riverbend.edu

Department *
Information Technology

Semester *
Semester 5

Course Selection * (choose at least one)

☒ ITA02 — Web Technology (4 cr) ☐ CSA05 — Data Structures & Algorithms (4 cr)
☒ CSE21 — Database Management Systems (4 cr) ☐ ITE14 — Cloud Computing Fundamentals (3 cr)
☒ CSL08 — Web Technology Laboratory (1 cr)

Submit registration **Clear form** Registration submitted — see your summary below.

Registration Summary

STUDENT NAME
Ananya Rao

REGISTER NUMBER
23ITR112

DEPARTMENT
Information Technology

SEMESTER
Semester 5

TOTAL COURSES
3

TOTAL CREDITS
9

Selected Courses

ITA02 — Web Technology	4 cr
CSE21 — Database Management Systems	4 cr

5.7 Debugging

During development, `console.log()` statements were used at two points in the submit handler: once when validation fails (logging the field values and the selected-courses array so a broken selector or empty checkbox group is easy to spot), and once after a successful submission (logging the final totals object). This surfaced an early bug where the course checkbox values used a comma inside a course name, which broke the pipe-delimited `split()` parsing — corrected by choosing a delimiter (`|`) that does not appear in any course name.

6. CSS Techniques Applied

- Box model: consistent padding/margin rhythm across cards, table cells, and form fields; border used for card outlines instead of shadows.
- Flexbox: header bar, hero content, checkbox group, and form action row.
- CSS Grid: two-column instructions/links section, registration form fields, and the summary key-value grid (auto-fit, minmax for responsive columns).
- Hover/focus states: navigation links, table rows, hyperlinks, and buttons all have distinct hover treatments; inputs show a focus ring for keyboard accessibility.
- Responsive design: a single media query at 780px collapses grid layouts to a single column and stretches buttons to full width on small screens.

7. Testing Summary

Test Case	Input	Expected Result	Status
Empty required fields	Submit with blank Register No./Name	Inline error shown, submission blocked	Pass
Invalid email	ananya.rao#riverbend	"Enter a valid email address" shown	Pass
Semester out of range	Semester left unselected	"Please select a semester" shown	Pass
No course selected	Submit with 0 checkboxes checked	"Select at least one course" shown	Pass
Valid submission	All fields valid, 3 courses selected	Summary panel shows totals: 3 courses, 9 credits	Pass

8. Conclusion

The completed portal satisfies every task requirement from the assignment: a semantically organised page with navigation and contact details, a five-row course table, instructional lists with internal/external hyperlinks, a validated registration form, client-side storage of selections as an array of objects, reusable calculation logic, and a dynamic, reload-free registration summary. The interface was also verified against a mobile breakpoint and checked in the browser console to catch and fix a data-parsing bug before final submission.

9. Rubric Self-Assessment Reference

The table below maps this report's sections back to the grading rubric for quick cross-checking during evaluation.

Criterion	Marks
HTML/XHTML structure and semantic organization	15
Lists, tables, hyperlinks and registration form	15
CSS design, layout and responsiveness	20
JavaScript validation and event handling	15
Functions, arrays/objects and credit calculation	15
Dynamic registration summary	10
Debugging, code quality and execution	10
Total	100